Multimodal operator-support prototype using industrial XR data

Xingdi Tong

1. Problem Formulation and Motivation

Human-centric manufacturing aims to support operators rather than replace them. Supporting an operator effectively requires first understanding their state, including what they are doing, how far they have progressed, and whether they are performing the task smoothly. Manual observation cannot provide this understanding in a continuous and objective way. The goal of this task is to estimate the state of the operator automatically from multimodal sensor data, providing a basis for supporting operators in manufacturing. The state of the operator in this context has two aspects: the objective progress of the task and the behavior of operator during working

The first aspect is task progress. We define it as the assembly state that depends on whether each part has been installed. If the installation status of every part is known at each frame, the assembly state is fully described. We therefore decompose the problem into predicting the binary installation status of each part at every frame. Once the installation status of all parts is known, the task progress follows directly from the number of installed parts, the workflow state is determined by their configuration, and transitions correspond to frames where this configuration changes. A single decomposition thus answers all three.

The second aspect is the behavior of operator during the task. Knowing the progress does not reveal whether the operator is carrying out a step smoothly. We therefore define an operator stall indicator that detects abnormal hesitation at the current assembly step, measured relative to the operator's own norm. A stall is reflected in the operator's behavior in three ways. The primary sign is duration, meaning the operator spends much longer on the current step than is normal for them. This is supported by two behavioral signals. One is the gaze-to-hand distance, since a struggling operator may look away from the area where the hands are working. The other is the hand motion energy, since the hands may stop or move in an unsettled way when the operator is struggling.

The indicator is expressed as a stall score S_t that accumulates over frames:

$$S_t = \min(1, \max(0, S_{t-1} + r_t))$$

The score is bounded between 0 and 1. The increment r_t is governed by duration as the master gate. When the current step lasts longer than the operator's own norm, r_t is positive and the score climbs; otherwise, r_t is negative and the score decrease back toward 0. Once a step is judged prolonged, the climbing rate is dependent on the two supporting signals, so that gaze drift or effortful hand motion makes the score rise faster. All judgements are made relative to the operator's own baseline rather than a fixed group. The detailed computation of r_t and the personal baselines is given in Section 3.2.

Together, the two aspects form a complete picture of the state of operator. The assembly state estimation describes the objective progress of the task, and the stall indicator describes how the operator is performing it.

2. Modality Selection and Justification

The HoloLens 2 provides four modalities. These are RGB video, eye gaze, hand joints, and head pose. We started from the motion modalities, which are hand, gaze, and head pose. Assembly is an active process, including reaching, grasping parts, and looking at the work area, so these modalities naturally reflect the assembly actions.

However, motion alone is not enough, since it only reflects the actions around the installation, which is an indirect cue. The assembly state is mainly a visual fact about objects. Whether a part is installed is shown most directly by how the scene looks. We therefore add the RGB modality to capture the visual state of the scene. We therefore use all four modalities. A single-modality comparison in Section 5 shows that RGB and hand are the stronger modalities while gaze and pose are weaker on their own, and that the differences between configurations are small given only four operators. We keep all four rather than dropping the

weaker ones for robustness. A single modality fails completely when its sensor drops out, whereas a multimodal model can keep estimating from the remaining modalities when one is missing.

3. Method

This section shows two parts as shown in the Figure 1. The first part is about the building classifier for assembly state estimation which includes features engineer, multimodal fusion, the second part define an install stall indicator during assembling and then justify based on features and evaluation metrics outputted in first part.

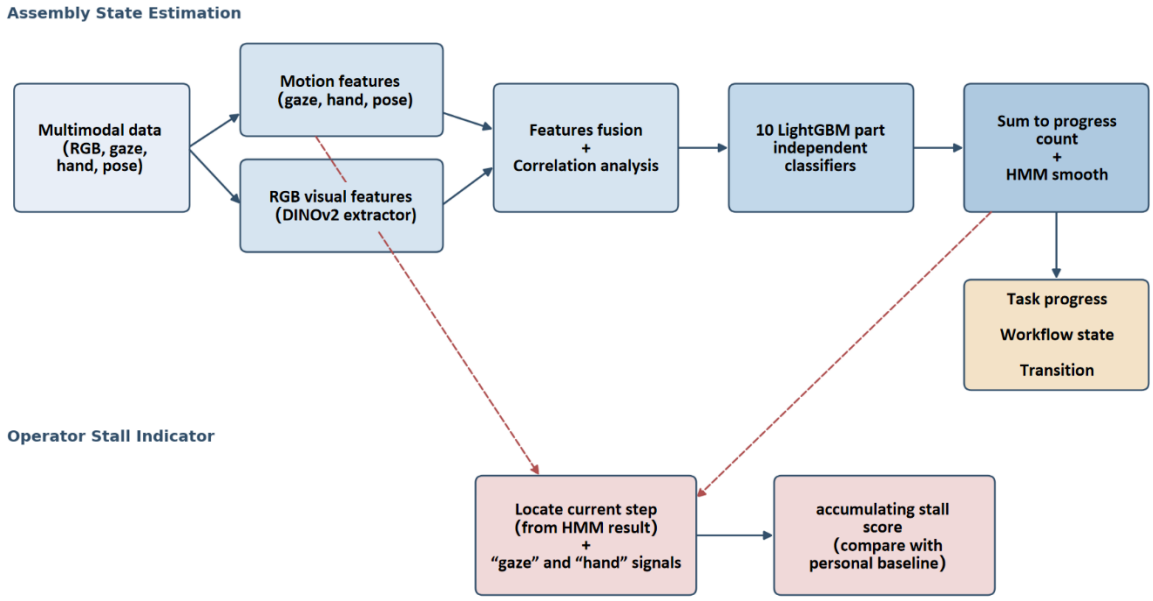

Figure 1. Overview of the prototype pipeline

3.1 Data

This work uses the IndustReal dataset(Schoonbeek et al., 2024), which provides egocentric recordings of an assembly task captured with a HoloLens 2. Each recording contains synchronized multimodal streams, including RGB video, eye gaze, hand joints, and head pose, together with PSR_labels that annotate the installation events of each part. The training set consists of recordings from four operators, and the test set is a separate recording (27_assy_0_1) from an operator not seen during training. All progress and part state ground truth is derived from the PSR_labels.

3.2 Assembly state estimation

3.2.1 Feature Engineering

Multimodal sensor data recorded by Hololens 2 captures different aspects of assembly process at each frame. Modalities differ in format. The Hand data records 2D coordinates of 26 hand joints per hand, while the RGB data is raw images. Based on the difference on format, we group input into two categories by how features are extracted.

The first category is hand-crafted statistical features from hand, gaze and pose stream. Installing a part requires operators to reach, grasp and place it, so the motion of hands changes in a characteristic way. We therefore extract action-level features such as hands speed, motion energy and the thumb-to-index distance that reflects grasping. Three design choices shapes these features. First, we extract action-level descriptors rather than raw coordinates, since the pixel position is uninformative for assembly state while the motion and grasp are. Second, we relabel two hands as dominant and non-dominant based on which hand has higher average motion energy across the recording. This removes the difference between left-handed and right-handed operators and helps model generalize across people. Third, all the features are aggregated over a sliding window of three seconds rather than a single frame, so each feature is stable summary of a short interval. Gaze and pose are treated the same way, since keeping the eye on the part and holding head steady are also informative. We extract the position and variability over the same window.

The second category is visual features from RGB. Simple statistical features cannot be hand-designed from raw image, and with only four operators there is too little data to train a vision model. Therefore, we adopt DINOv2(Oquab et al., 2024), a pretrained vision model, as a feature extractor without fine-tuning, to extract 384-dimensional feature embeddings from each frame.

3.2.2 Multimodal Fusion

As defined in Section 2, we describe each part with a binary indicator where 0 and 1 represent uninstalled and installed, respectively. A full state is a 11-bit code, and the progress is the total number of ones. We then model this as multi-label classification, with one binary classifier per part, rather than a single multi-class classifier. The reason is that the parts are not mutually exclusive. They are independent states that coexist, so the multi-label formulation is more faithful to the task. In practice, the base part is always present, so the its bit is constantly 1,and we train classifier only for the remaining ten parts.

Deep learning method is common for sequence classification, but with only four operators it would overfit. We therefore use the LightGBM as the part classifiers, a gradient-boosted tree model that trains decision trees sequentially, each correcting the residuals of the previous. It is well suited here because the features are heterogeneous in scale, contain missing values, and the dataset is small. LightGBM handle these without feature normalization, and is efficient and easy to interpret.

We then adopt the early fusion strategy, which means the four modality features are feed together into each classifier, since each part status classification must consider all features. In addition, as a standard preprocessing step, we apply a correlation analysis on the training data and drop one feature from any pair whose absolute correlation exceeds 0.9, removing redundancy without using labels. On this data the effect is minor, removing only a few redundant motion features, since the DINOv2 embedding dimensions are already compact and show little redundancy. We keep this step as routine practice rather than a major component.

3.2.3 Temporal Smoothing

The ten classifiers are trained so independently that they ignore the temporal continuity of data. This cause a problem.The predictions of progress at each frame fluctuate strongly. However, it is physically impossible that installed parts disappear in the next frame. We therefore we add the Hidden Markov Model (HMM) on the top of the classifier. We sum the ten binary predictions into a progress count at each frame and then introduce the HMM smoothing this sequence data. The hidden state are the progress level from 0 to 10. The transition matrix allow only the state staying same state or moving forward, and forbid going backward. Viterbi method then applied to find the most likely path that is consistent with the predicted progress at each frame. The contribution of HMM here is correcting fluctuation and keep the temporal continuity of prediction rather than interfering the prediction results from classifiers.

3.2.4 Evaluation Metrics

The evaluation metrics correspond to two aspects of task defined in Section 3.2.2. The ground truth is from the PSR_labels , which record the installation event of each part. Since a part remains installed once mounted, its installation status is set to 1 from the installation frame onward. The true progress at each frame is then the number of installed parts.

The first aspect is task progress. We introduce Mean Absolute Error (MAE), which is the absolute value of the difference between the true value and the predicted value about progress, to evaluate how many parts have been installed at each frame i:

$$MAE = \frac{1}{T}\sum_{i=1}^{T}\left|X_{tr,i} - X_{pred,i}\right|$$

where T is the total number of frames, $X_{tr,i}$ is the true number of installed parts at frame i, and $X_{pred,i}$ is the predicted number after HMM smoothing.

The second aspect is per-part installation state. We use Macro-$F1$ to evaluate whether each specific part is correctly identified as installed or not. For each part k, a prediction can fall into one of three cases: a true positive (*TP*, the part is installed and correctly predicted as installed), a false positive (*FP*, the part is not

installed but predicted as installed), or a false negative (*FN*, the part is installed but predicted as not installed). *F*1 combines precision and recall into a single score:

$$F1_k = \frac{2 \times Precision_k \times Recall_k}{Precision_k + Recall_k}$$

Where, $Precision_k = \frac{TP_k}{TP_k + FP_k}$, and $Recall_k = \frac{TP_k}{TP_k + FN_k}$.

Macro-F1 is the average across all ten parts:

$$\text{Macro-}F1 = \frac{1}{10}\sum_{k=1}^{10} F\,1_k$$

This reflects our interest in whether each individual part can be correctly identified.

3.2.5 Train and Test

The final model is trained on all available training recordings. Correlation-based redundancy removal is applied to the training data before fitting, and the resulting feature set is then applied to the test data. This ensures that no information from the test set influences feature selection.

During training, the key hyperparameters of LightGBM are num_leaves, n_estimators, and min_child_samples, which control model complexity, ensemble size. These are not tuned. With only four operators, tuning would risk overfitting to specific individuals. To verify that the chosen values are appropriate, we conduct a sensitivity analysis under leave-one-operator-out cross-validation (LOGO-CV). In each fold, one operator is held out for evaluation and the remaining three are used for training. This is repeated four times, once for each operator. Within each fold, correlation analysis and redundancy features removal are performed on the training operators only, and the held-out operator is used solely for evaluation. We then vary one hyperparameter at a time while keeping the others fixed. The results are shown in Table 1.

Table 1. Hyperparameter sensitivity analysis

Hyperparameter	Value	MAE	Macro-F1
num_leaves	15	1.02 ± 0.07	0.62 ± 0.02
	31*	1.04 ± 0.12	0.62 ± 0.01
	63	1.14 ± 0.03	0.62 ± 0.01
n_estimators	150	1.04 ± 0.12	0.62 ± 0.01
	200*	1.04 ± 0.12	0.62 ± 0.01
	300	1.08 ± 0.09	0.62 ± 0.01
min_child_samples	20	1.04 ± 0.03	0.62 ± 0.01
	30*	1.04 ± 0.12	0.62 ± 0.01
	50	1.02 ± 0.15	0.62 ± 0.01
* Values used in the final model.			

MAE varies by at most 0.12 across each sweep, comparable to the across-fold standard deviation of 0.07-0.15. Macro-*F*1 remains constant at 0.62 throughout. This confirms that the model is insensitive to these hyperparameters and that the chosen values lie within a stable region.

The test set is used solely for final evaluation and plays no role in training or feature selection. The trained model is applied to the test recording to produce predictions at frame, which are then evaluated using the metrics defined in Section 3.2.4.

3.3 Operator Stall Indicator

Based on the assembly state estimate, the way to measure the operator stall indicator requires the HMM-smoothed progress to locate the current step, and reuses the gaze and hand motion features from the

same pipeline. The stall score S_t is computed fully online and is strictly causal, which means frame t uses only information from before t. All baselines are personal, estimated from the operator's own past frames rather than a group.

The increment r_t in the accumulation $S_t = \min(1, \max(0, S_{t-1} + r_t))$ is decided by duration first, then modulated by the two supporting signals:

$$r_t = \begin{cases} -0.05, & \text{normal pace (duration} \leq 1.75 \times \text{personal mean)} \\ +0.016, & \text{prolonged, with gaze drift} \\ +0.008, & \text{prolonged, with abnormal high hand effort} \\ +0.004, & \text{prolonged, with neither} \end{cases}$$

When the current step stays within the normal pace of operator, the score decreases toward 0. Once the step is prolonged, the score rises, and the rate depends on the two supporting signals. Both are judged against the own history of operator up to the current frame, using the 85th percentile of that past values as the threshold. Gaze drift means the smoothed gaze-to-hand distance at the current frame exceeds the 85th percentile of past distances, which suggests stopping and searching and drives the fastest climb. High hand effort means the hand motion energy exceeds the 85th percentile of past energy values, which suggests physical struggle and drives a moderate climb. A step that is prolonged but shows neither signal still climbs slowly, as a safety net, so that a quiet but stalled step is not missed. In addition, these rates correspond to physical time at the 10 FPS recording rate. For example, at +0.016 the score reaches the 0.8 threshold after about 50 frames, about 5 seconds of continuous prolonged behavior.

4. Addressing Imperfect and Limited Data

Real factory data is noisy, incomplete, and varies between operators. We address this in several ways. When gaze, hand, or head pose is unavailable, the frame is marked as missing so it does not pollute the derived features. The training data is limited to four operators. Tuning hyperparameters on so few individuals would overfit, so we keep fixed settings and verify with a sensitivity analysis that the model is insensitive to them. For the same reason, DINOv2 is used frozen without fine-tuning, since fine-tuning a vision model on four operators would overfit.

Operators differ from one another. We reduce this variation by relabelling the two hands as dominant and non-dominant, which removes the difference between left-handed and right-handed operators. We also evaluate with leave-one-operator-out cross-validation, so the reported performance reflects generalization to operators the model has not seen.

Keeping all four modalities also provides robustness when an entire modality drops out. The model can continue to estimate from the remaining modalities rather than failing, which a single-modality model could not do.

5. Results and Evaluation

5.1 Results

5.1.1 Assembly State Estimation

On the test recording, the model predicts task progress with a MAE of 1.30 before smoothing and 1.07 after HMM smoothing, confirming that the temporal smoothing reduces fluctuations and improves the progress estimate. The Macro-$F1$ over the ten parts is 0.53. The predicted progress closely follows the true progress over the full sequence, as shown in Figure 2, where the raw prediction fluctuates and the HMM-smoothed curve is monotonic and stable.

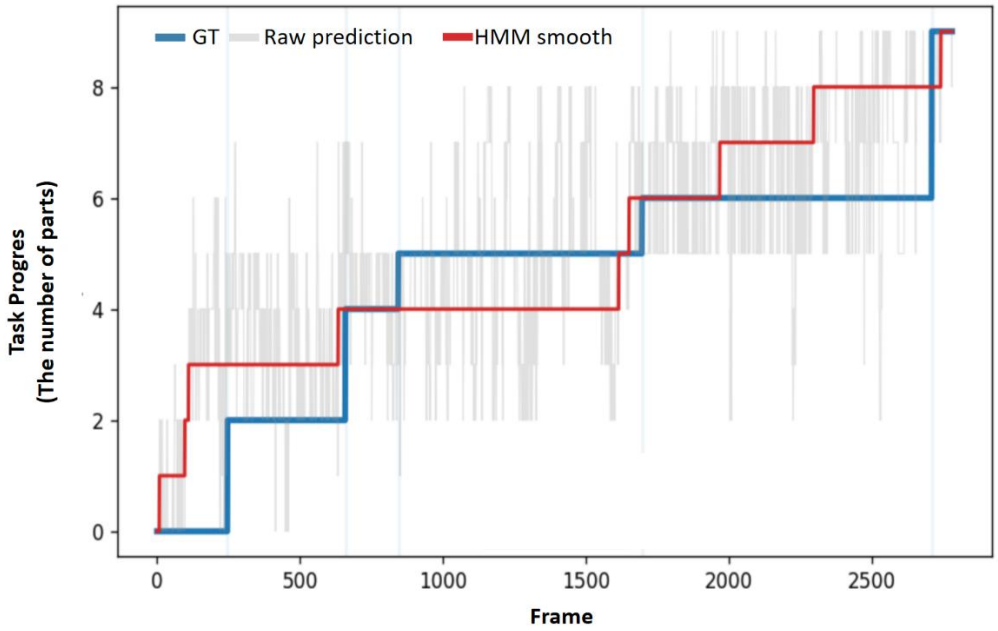

Figure 2. Predicted versus true task progress on the test recording.

The modality comparison is shown in Table 2. RGB and hand are the two stronger modalities. Hand alone gives the lowest progress MAE, and RGB alone gives the highest Macro-*F*1, which reflects that hand motion is closely tied to how many parts are installed, while the visual appearance is more informative about which specific part is installed. Gaze and pose are much weaker on their own, with pose carrying almost no discriminative power. The differences between the combined modalities are small given only four operators. We keep all four modalities rather than dropping the weaker ones.

Table 2. Comparison of Modality based on Evaluation Metrics

Modality	**MAE**	**Macro-*F*1**
RGB only	0.813376483	0.589231665
Gaze only	1.725997843	0.392941841
Hand only	0.726357425	0.502502559
Pose only	3.413879899	0.163209961
Hand + RGB	1.045667026	0.602834668
All four	1.065084502	0.528763713

The part results are uneven. The early large parts are recognized well, with *F*1 above 0.85 for the front and rear chassis and their pins. The late small parts are much weaker, with *F*1 around 0.14 to 0.24 for the brackets, screws, and wheel assemblies, and short rear chassis has an *F*1 of 0, since it has no positive frames in this test recording. The confusion matrices for each part in Figure 3 show two patterns. The early parts (the chassis and pins) reach high *F*1, mainly because they have a large number of positive frames, so even with a few hundred false positives or false negatives the *F*1 stays high. The late small parts have few positive frames and show clear directional errors. The brackets and screws are over-predicted, and the rear wheel is under-predicted. A likely reason is that the DINOv2 feature is a single global descriptor of the whole frame, so small late parts that barely change the scene are hard to detect.

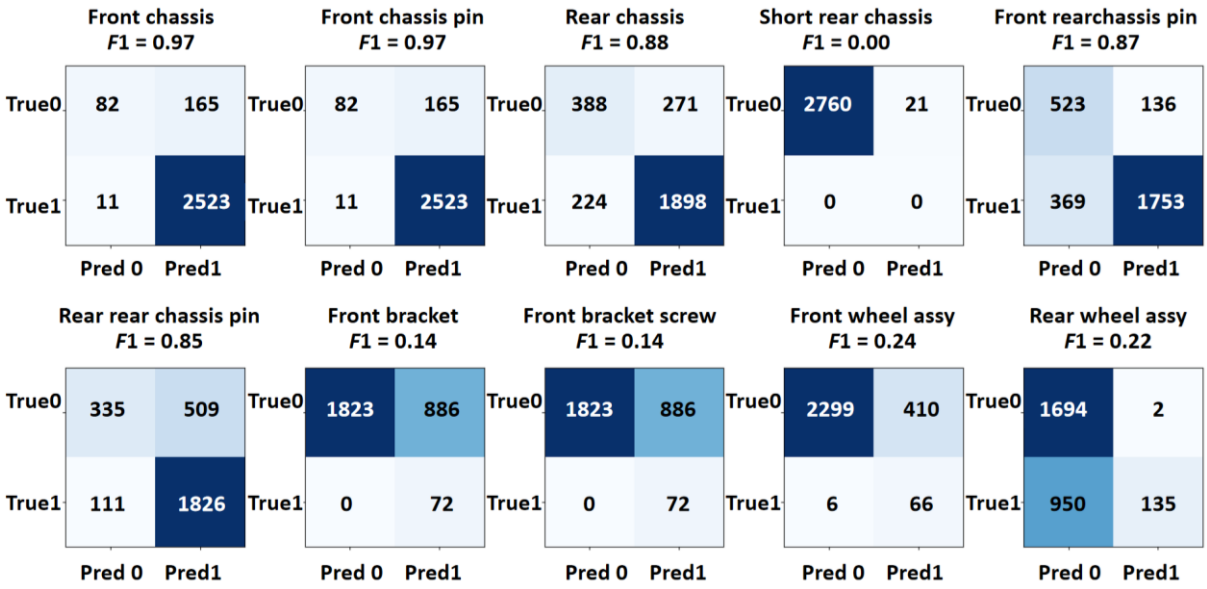

Figure 3. The confusion matrices on the test recording for each part

To examine what the classifiers rely on, we aggregate the LightGBM feature importances over the ten part models, shown in Figure 4. RGB contributes about 66% of the total importance and the motion features about 34%. This is consistent with the modality comparison, where RGB was the strongest single modality, while confirming that the motion features still carry a substantial share rather than being redundant. Among the motion features, the most important ones are interpretable action cues, including the spread of the dominant hand, the head orientation, the non-dominant pinch, and the distance between the two hands. This is the benefit of the hand-crafted motion features. Unlike the 384 opaque RGB dimensions, these features let us read which physical cues the model uses, which supports the interpretability of the approach.

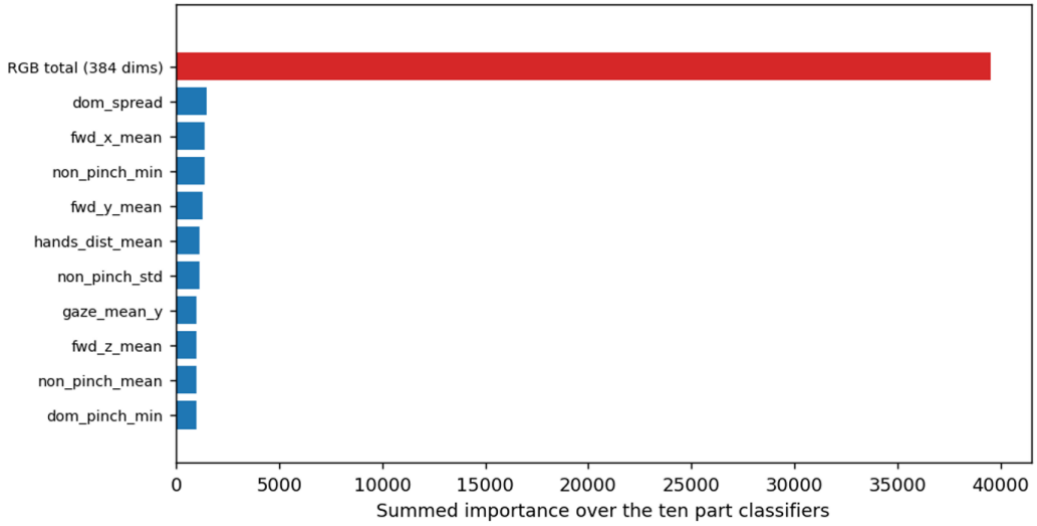

Figure 4. Feature importance summed over the ten part classifiers, only the ten most important motion features are shown.

5.1.2 Operator Stall Indicator

The stall score over the test recording is shown in Figure 5, together with the true and predicted progress and the independent validation signals. In normal assembly, where progress advances at a steady pace, the score stays near 0 and does not raise false alarms. The score rises and crosses the 0.8 threshold in two segments, both of which correspond to places where the progress stays flat for an unusually long time. This matches the intended behavior, since the indicator is designed to react to prolonged stalls relative to the operator's own pace, not to normal steps.

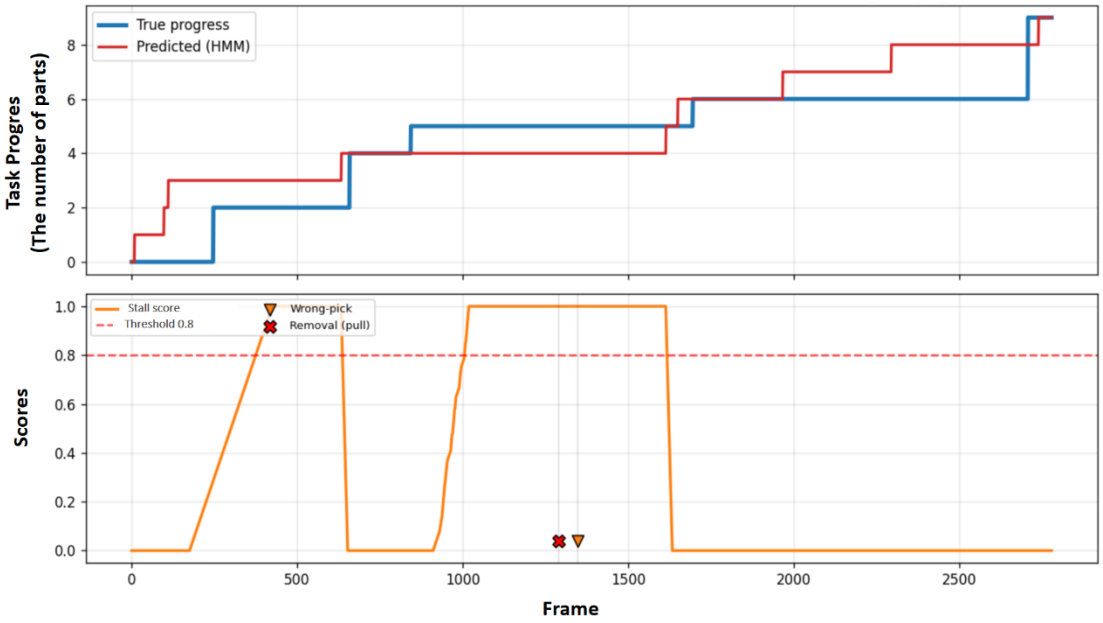

Figure 5. Stall score over the test recording, with the predicted and true progress above

The stall indicator detects prolonged hesitation, but there is no direct ground truth for when an operator is actually struggling. We therefore validate it against independent signals that objectively indicate difficulty. One such signal is correction behavior. The AR_labels provide low-level actions such as take, put, and pull, but not correction events directly, so we translate them. A put that closely follows a take of the same part is treated as a wrong-pick, where the operator picked a part and put it back without installing it. A pull is treated as a removal, where an installed part is taken off. Both indicate that the operator made a mistake and had to correct it, which is an objective sign of difficulty at that step. Since the scoring never uses these events, they serve as an independent check. In this recording, the one wrong-pick and one removal both fall inside a long flat segment where the score is at its maximum, which means the indicator flags the period in which the operator was struggling and correcting.

5.2 Strengths and Limitations

The two tasks together build a layered understanding of the state of operator . The assembly state estimation captures what the operator is doing and how far the task has progressed, and the stall indicator goes one step further to capture how well the operator is carrying it out. The indicator does not take any actions. It provides an understanding of the state of operators that can serve as a basis for later support decisions, which keeps the system informative without forcing a fixed response.

Several design choices give the approach its strengths. Decomposing state recognition into independent part binary classifiers, rather than a single multi-class classifier, makes better use of the limited data, since every frame contributes to every part and unseen part combinations can still be expressed. The HMM gives the predictions temporal continuity and enforces monotonic, physically valid progress. LightGBM keeps the classifiers lightweight and interpretable on a small, heterogeneous feature set. For the stall indicator, accumulating a score over frames rather than making an instantaneous decision encodes how long a stall has lasted and stays robust to momentary fluctuations.

It also has clear limitations. First, some small parts are estimated poorly, because the visual feature is a single global descriptor of the whole frame and is insensitive to parts that occupy a small region. Using region level or part localized visual features would likely help. Second, the two pairs of a main part and its fastener share identical labels and are always installed together, so modelling them with separate classifiers is redundant. These could be merged into a single target. Third, the stall indicator detects that a stall occurs without covering other operator states such as fatigue. To combine it with additional signals could separate the cause and extend its scope. Additionally, the stall thresholds are set by assumption rather than calibration, since stalling has no ground truth here. Finally, the assembly state estimation is currently offline, since the HMM

smoothing needs the complete sequence, so it does not yet run in real time. These limitations mainly are from the limited data and the lack of stall ground truth, and they define clear directions for future work as more data becomes available.

6. Reflect on practical relevance

The two outputs map naturally on three systems in human-centric manufacturing. The assembly state estimate can feed a Human Digital Twin. The Human Digital Twin needs a live representation of what the operator is doing, and the task progress and workflow state provide exactly the status of part and the state of operator, updated continuously from sensor data.

The same state estimate supports an XR guidance interface. To show the right instruction at the right moment, the interface needs to know which step the operator is currently on. The estimated progress and workflow state can drive this, so that guidance advances in step with the operator rather than on a fixed script.

The stall indicator fits an operator-support system. When the score crosses its threshold, it signals that the operator has been stuck at a step longer than their own norm, which can trigger support such as an XR hint for the current step or a notification to a human supervisor or an automated assistant. Because the indicator only reports the state of operator and does not take any actions, the decision of whether and how to intervene stays with the support system, which keeps the operator in control.

One practical consideration is real-time operation. The two outputs share the same bottleneck, which is the HMM smoothing in the assembly state estimation. The HMM currently uses Viterbi decoding over the complete sequence, so it needs the whole recording before it can resolve the progress path, which makes the assembly state estimation offline. The stall indicator depends on this smoothed progress to locate the current step, so it has the same offline limitation. Therefore, for live use with a connected device, the HMM can be switched to online forward decoding, which outputs the most likely state at each frame using past frames. Once the progress is produced online in this way, the stall indicator becomes online as well. This enables real-time operation for both outputs, at the cost of slightly weaker smoothing.

References

Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M., Sharma, V., … Bojanowski, P. (2024). DINOv2: Learning Robust Visual Features without Supervision (arXiv:2304.07193). arXiv. https://doi.org/10.48550/arXiv.2304.07193

Schoonbeek, T. J., Houben, T., Onvlee, H., De With, P. H. N., & Van Der Sommen, F. (2024). IndustReal: A Dataset for Procedure Step Recognition Handling Execution Errors in Egocentric Videos in an Industrial-Like Setting. 2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 4353–4362. https://doi.org/10.1109/WACV57701.2024.00431